

(underlined bits are in error):

01110111 --- Random error (errors are distributed and in second and sixth bit locations),

and

01101101 --- Burst error (errors are clustered on fourth, fifth and sixth bit locations).

Several EDCs and ECCs are used to address both random and burst errors. R.W. Hamming introduced one-bit error-correcting codes in 1950. (7,4) and (13,8) are the examples of one-bit ECCs. P. Elias developed convolution codes in 1955. In 1959, R.C. Bose and D.K. Chaudhuri proposed multiple error-correcting codes. These are very powerful codes and known as generalised Hamming codes. A. Hocquenghem independently designed the codes proposed by Bose and Chaudhuri. That is why these codes are known as BCH codes. In 1960, I.S. Reed and G. Solomon designed powerful block codes particularly for burst errors, known as Reed Solomon codes. In 1960, G.D. Forney introduced the concept of concatenated codes. In 1967, A.J. Viterbi introduced an important convolution code known as Viterbi code. Turbo code, low-density parity code, combined turbo code, punctured turbo code, cyclic redundancy code (CRC) and Golay code are the other important codes.

Error detection and correction codes begin with parity codes. Parity codes are EDCs. Parity bit is used for error detection. In parity codes, a parity bit (either odd or even) is appended to the original message bits; parity bit is the redundant check bit. Even parity bit ensures even number of 1's in the code (message plus parity bit). Odd parity ensures odd number of 1's in the code. If an original message of seven bits is 1110001, its codes with parity bit are: even parity [11100010] and odd parity [11100011], where bold bits are parity bits.

Hamming code is an ECC. Richard Hamming, a theorist with Bell Telephone Laboratories in the 1940s, developed the Hamming code method of error correction in 1949. The key to

the Hamming code is the use of extra parity bits to allow identification of the errors. Hamming code (7,4) can detect and correct one-bit error, whereas (13,8) code can detect up to two simultaneous bit errors, and correct single-bit errors. A code with this ability to reconstruct the original message in the presence of errors is known as the error-correcting code. By contrast, the simple parity discussed above cannot correct errors, and can detect only an odd number of errors.

An ECC always has more check bits than EDC and hence requires more bandwidth. In simple parity check bit is just one, whereas in (7,4) and (13,8) codes check bits are 3 and 5, respectively. Code capability and complexity in system design are the other parameters for selection of a code for particular applications.

Encoding of (7,4)

Hamming code (7, 4) encodes four bits of data into seven bit blocks called 'code word.' The extra three bits are parity bits. Each of the three parity bits maintains even parity for three of the four data bits, and no two parity bits are for the same three data bits. If the four data bits in (7, 4) are d_1, d_2, d_3 and d_4 , Hamming code parity bits p_1, p_2 and p_3 are calculated as:

$$p_1 = d_2 + d_3 + d_4$$

$$p_2 = d_1 + d_3 + d_4$$

$$p_3 = d_1 + d_2 + d_4$$

where '+' means bit-wise exclusive OR operation, i.e., sum ignoring carry. For example, you can encode data 1010 using the Hamming code as 1011010.

Decoding of (7,4)

In a world without errors, decoding a Hamming code word would be very easy. Just leave out the parity bits. In the example of code word, the parity bits are 101 and when you leave these out, you will receive data bits as 1010.

But what if you receive a code word with an error and one or more of the parity bits are wrong? Suppose the received code word is 1011011. The first step is to check the parity bits to determine whether there is an error.

Calculate parity bits with received bits as:

$$p_1 = d_2 + d_3 + d_4 = 0 + 1 + 1 = 0$$

$$p_2 = d_1 + d_3 + d_4 = 1 + 1 + 1 = 1$$

$$p_3 = d_1 + d_2 + d_4 = 1 + 0 + 1 = 0$$

In this case, every parity bit is wrong. p_1, p_2 and p_3 should have been 010, but you received 101. Compare received parity bits with these calculated parity bits to get bit pattern 111. This bit pattern has decimal value of 7. Now reverse the bit at seventh position of the received code to get 1011010, and then leave out parity bits 101 to receive the corrected data as 1010.

Generalised Hamming code for single bit-error correction

Illustration of (7,4) Hamming code paves the way for explaining generalised Hamming code. In generalised code, coding is done as below:

1. All bit positions that are powers of 2 will be locations for parity bits in code words. Thus in code words, locations of parity bits are 1, 2, 4, 8, 16, 32, 64, etc.

2. All other bit positions are for the given original data to be encoded. That means data locations are 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 17, etc.

3. Each parity bit calculates the parity for some of the bits in the code word. The position of the parity bit determines the sequence of bits that it alternately checks and skips as follows: (i) Location 1: check 1 bit, skip 1 bit, check 1 bit, skip 1 bit, etc (1, 3, 5, 7, 9, 11, 13, 15,...), (ii) Location 2: check 2 bits, skip 2 bits, check 2 bits, skip 2 bits, etc (2, 3, 6, 7, 10, 11, 14, 15,...), (iii) Location 4: check 4 bits, skip 4 bits, check 4 bits, skip 4 bits, etc (4, 5, 6, 7, 12, 13, 14, 15, 20, 21, 22, 23,...), (iv) Location 8: check 8 bits, skip 8 bits, check 8 bits, skip 8 bits, etc (8-15, 24-31, 40-47,...), (v) Location 16: check 16 bits, skip 16 bits, check 16 bits, skip 16 bits, etc (16-31, 48-63, 80-95,...), (vi) Location 32: check 32 bits, skip 32 bits, check 32 bits, skip 32 bits, etc (32-63, 96-127, 160-191,...).

4. Set a parity bit to 1 if the total number of 1's in the positions that it